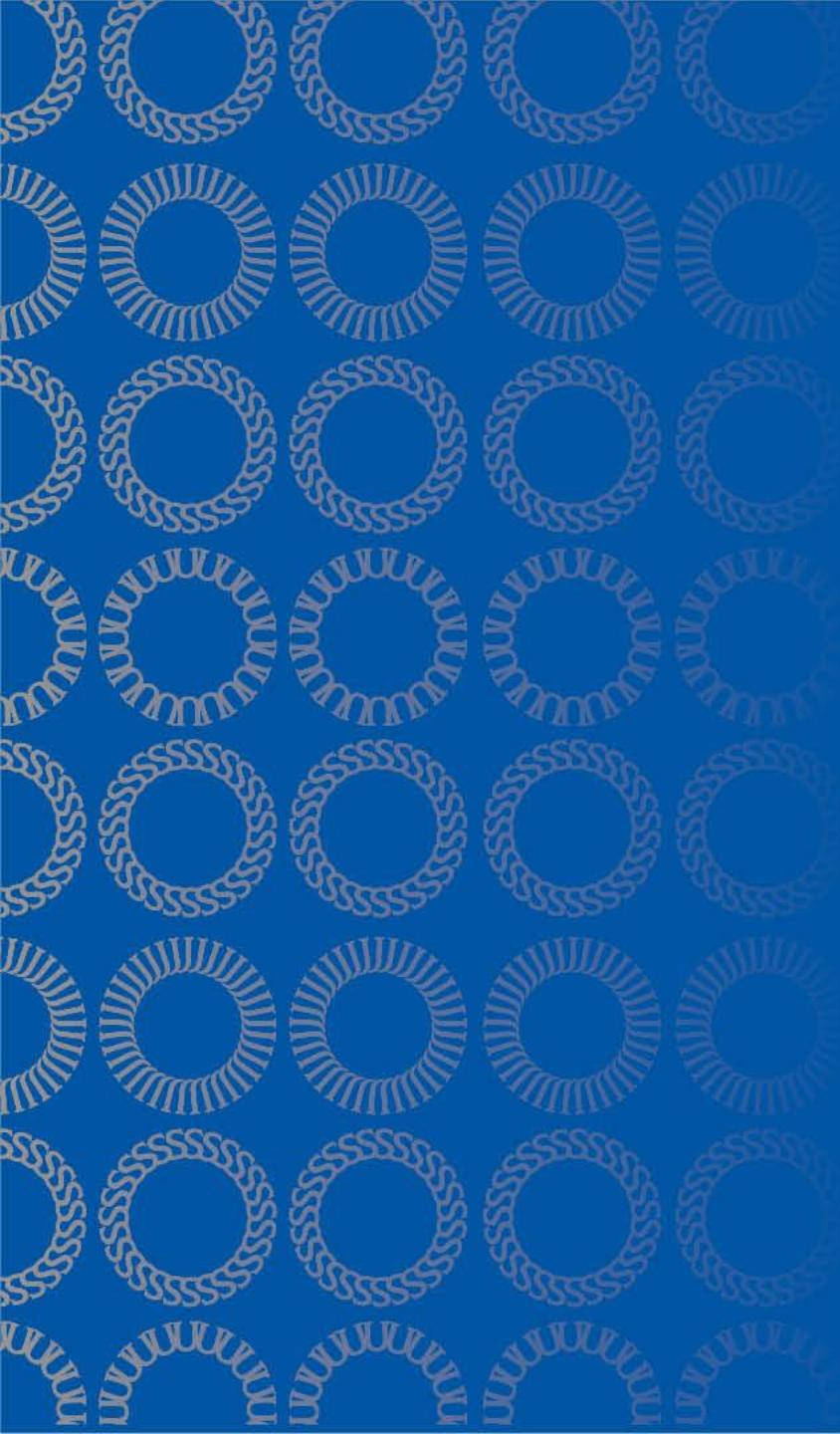




DATA 255 Deep Learning Technologies

Aug 28, 2025

Interdisciplinary Science 870

Taehee Jeong, Ph.D.

Deep Learning Technologies

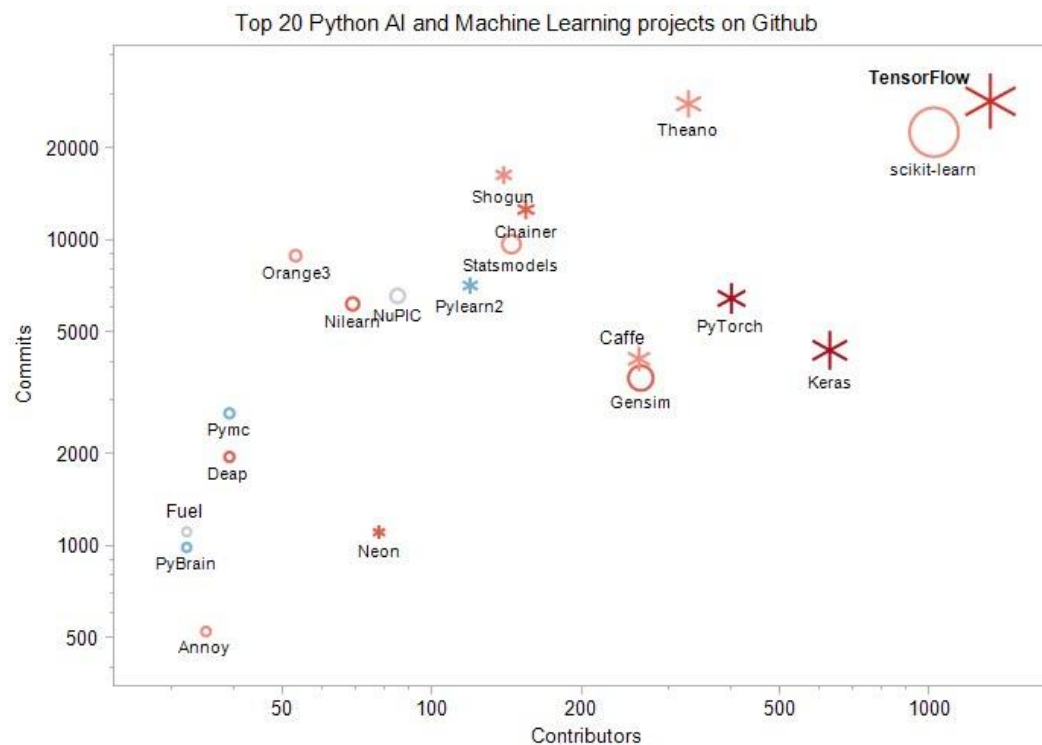
Study deep learning theory, techniques and technologies implement neural networks using deep learning frameworks to solve problems in various application domains

- Fundamentals of Neural network
- Deep neural network (DNN)
- Convolutional neural networks (CNN)
- Generative adversarial networks (GAN)
- Diffusion models

Today's Topics

- Deep Learning Framework
- Generative AI
- Deep neural networks

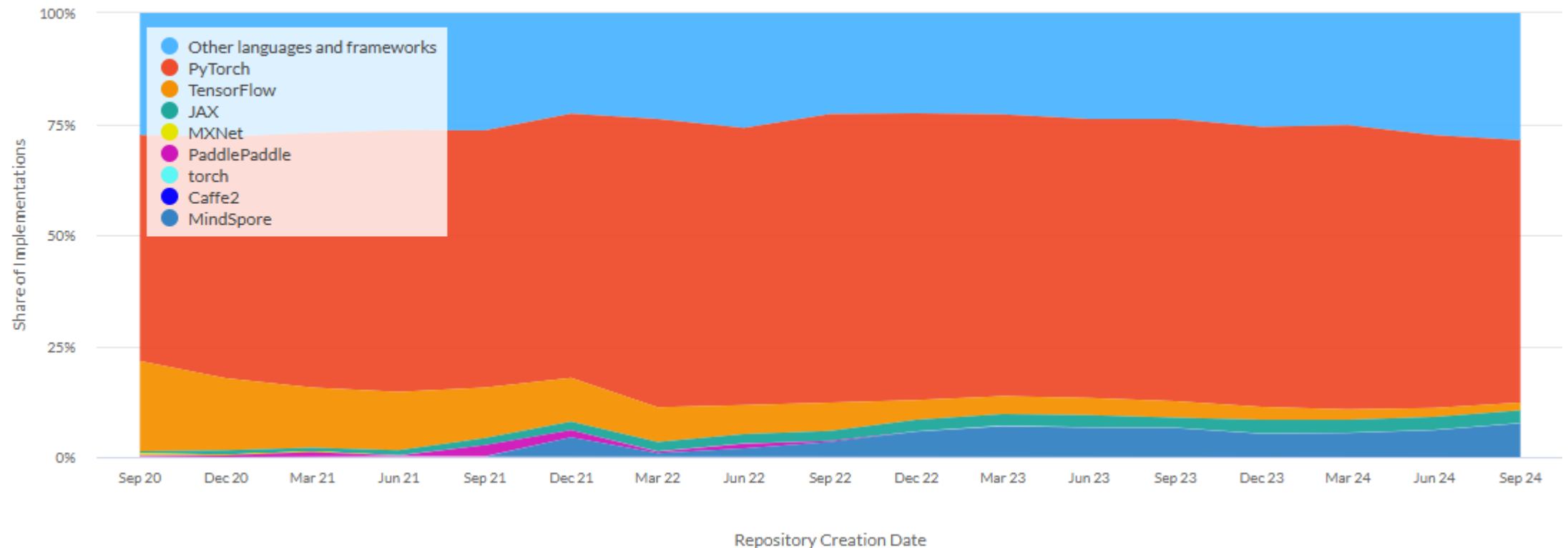
Deep learning Framework



Top 20 Python AI and Machine Learning Open Source Projects (2018)
By Ilan Reinstein, KDnuggets

Deep learning Framework

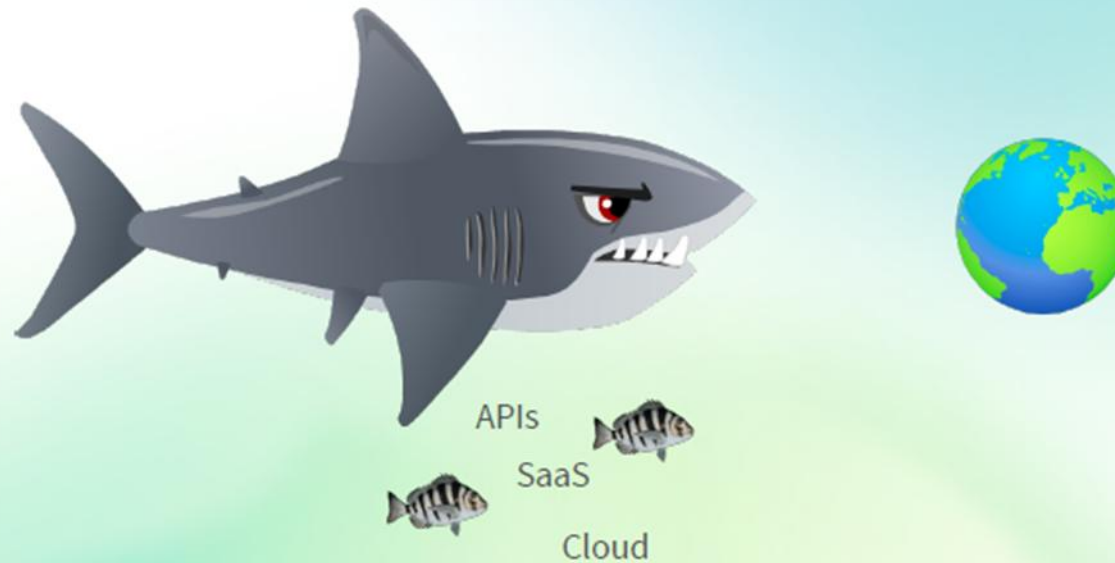
Paper Implementations grouped by framework



Source: <https://paperswithcode.com/trends>

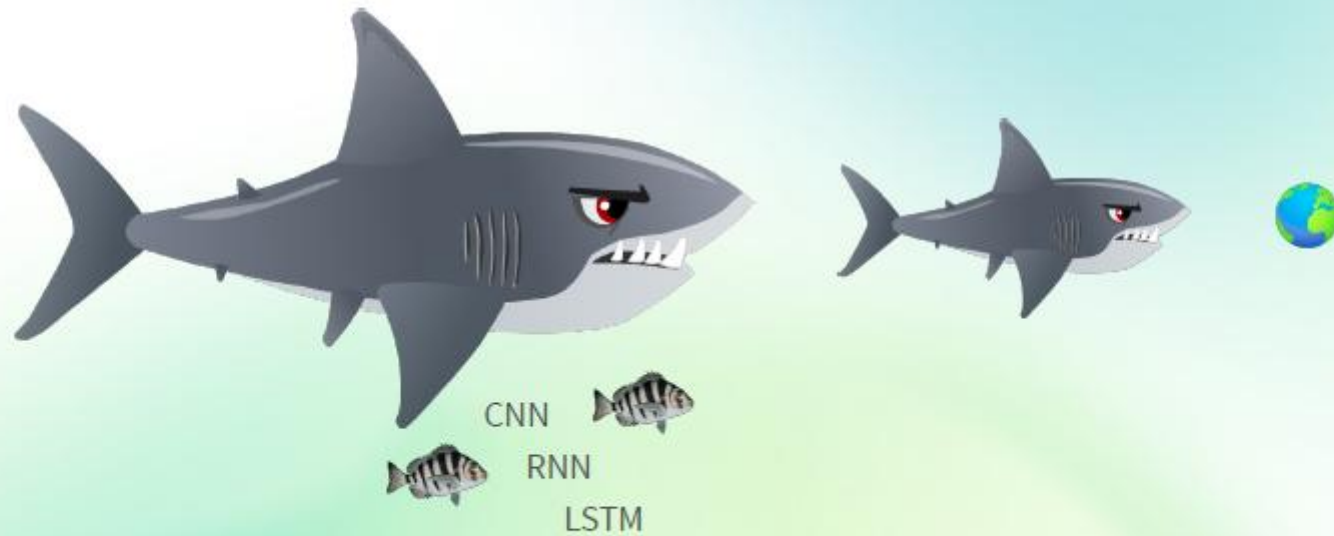
2011: Software is eating the world

<https://a16z.com/2011/08/20/why-software-is-eating-the-world/>



2017: Deep Learning is eating Software

<https://www.technologyreview.com/2017/05/12/151722/nvidia-ceo-software-is-eating-the-world-but-ai-is-going-to-eat-software/>



2022: Transformers are eating Deep Learning

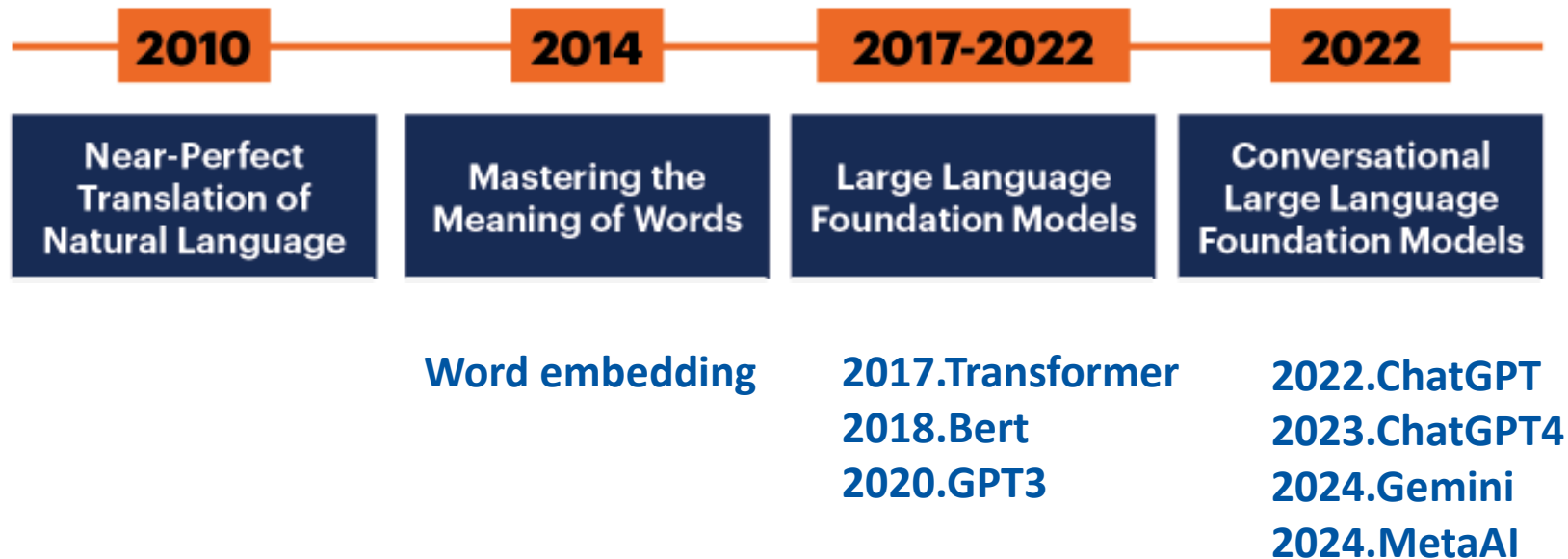


Generative AI

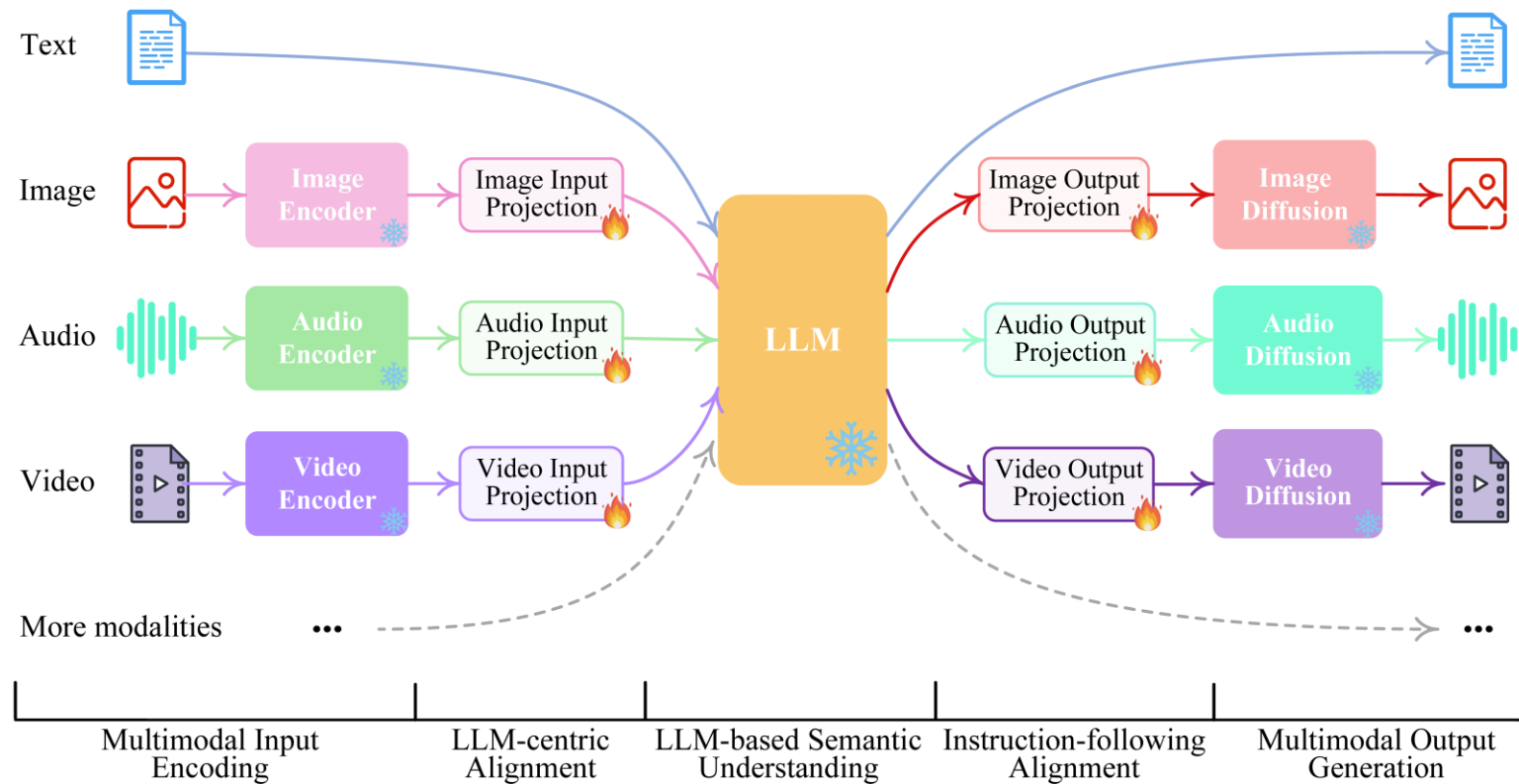
Generative AI can learn from existing artifacts to generate new, realistic artifacts (at scale) that reflect the characteristics of the training data but don't repeat it.

It can produce a variety of content, such as **images, video, music, speech, text, software code and product designs**.

The Journey to Generative AI



Multimodal-LLMs



Source from <https://github.com/NExT-GPT/>

Text-image-generation

(Closed source)

- DALL·E 3
- Imagen 2.5

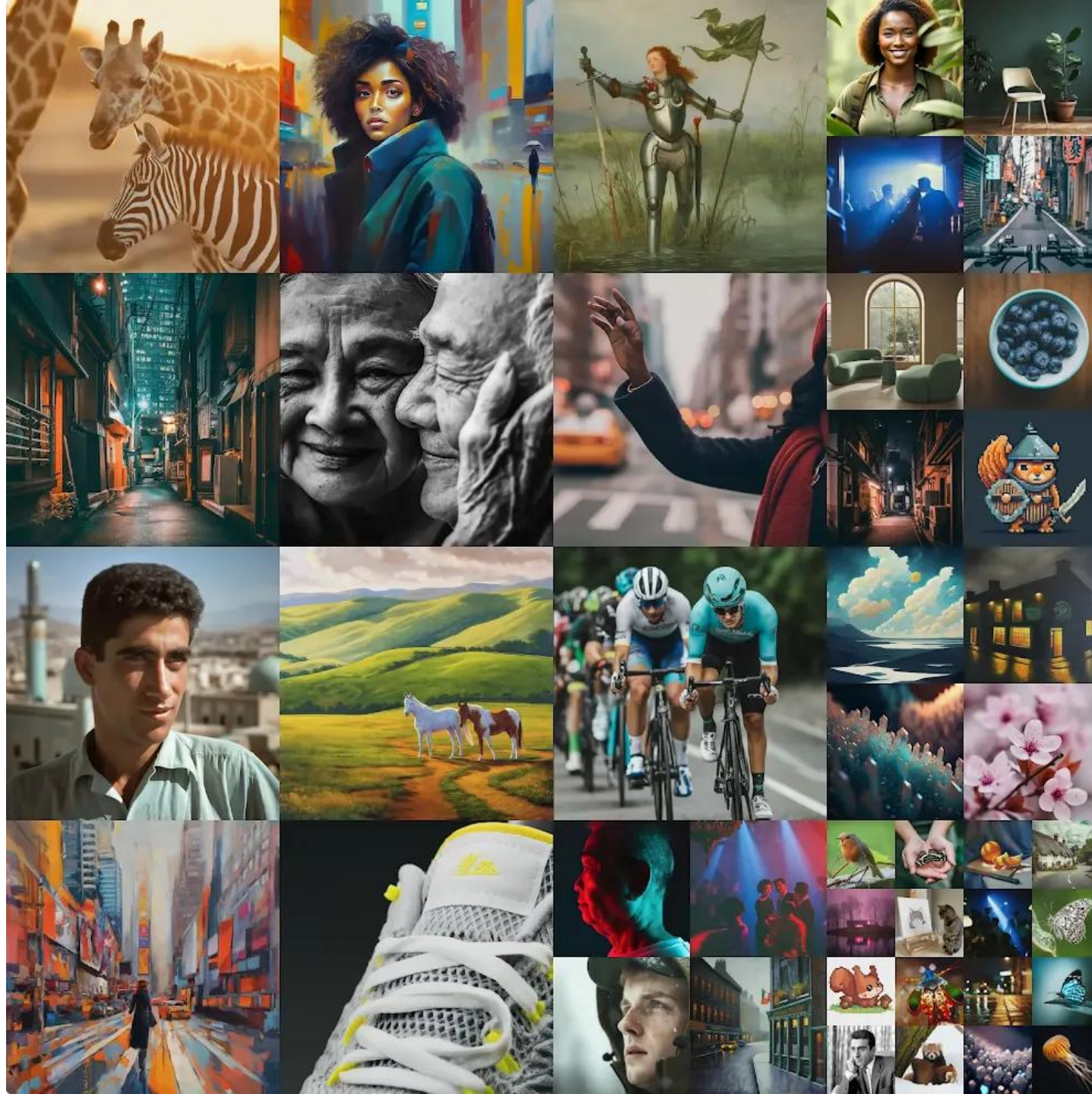
(Open source)

- Stable Diffusion 3

DALL·E 3: text-to-image synthesis



imagen2

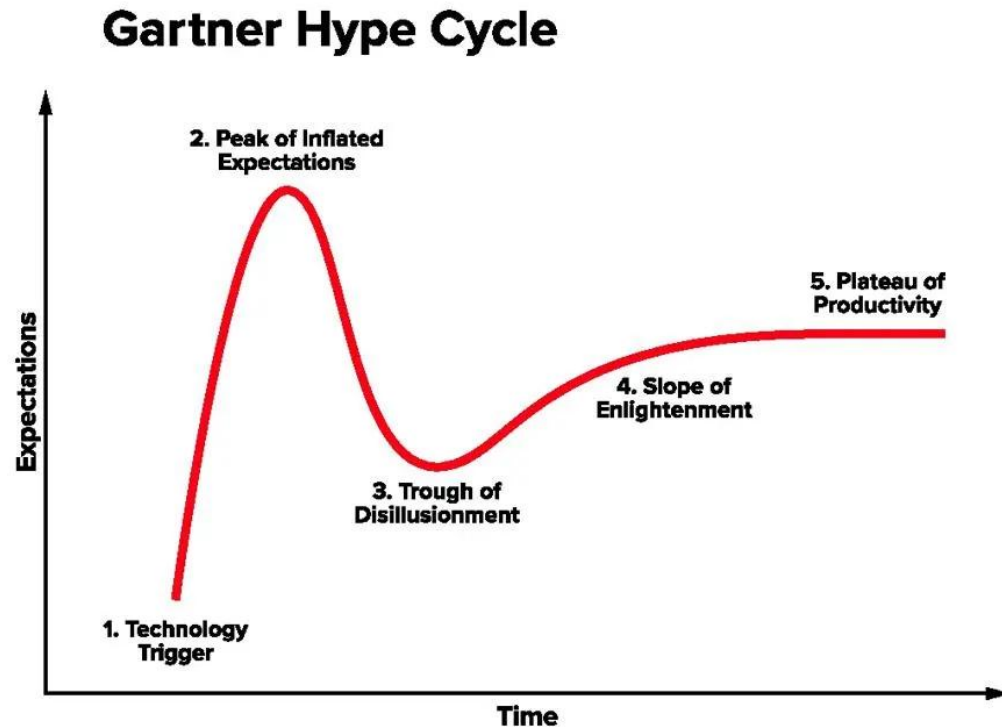


Diffusion :(Closed source) google

Stable diffusion 3



Gartner Hyper cycle



Technology Trigger:

A potential technology breakthrough sparks interest and early publicity.

Peak of Inflated Expectations:

Early publicity leads to some successes, but also to failures and unmet expectations.

Trough of Disillusionment:

Interest wanes as early adopters experience performance issues and low returns on investment.

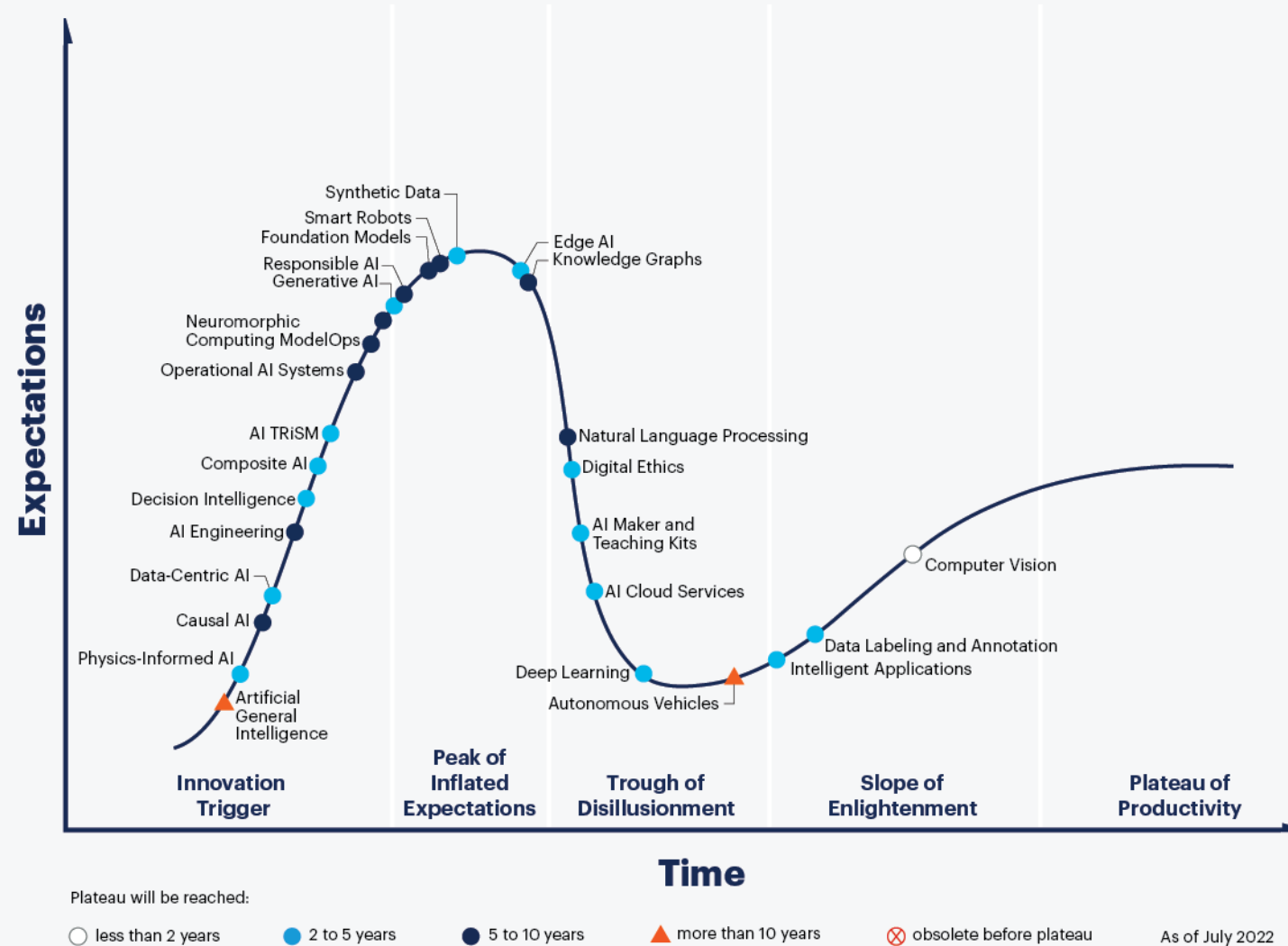
Slope of Enlightenment:

Early adopters begin to see benefits, and new applications emerge as second and third-generation products appear.

Plateau of Productivity:

The technology becomes mainstream, and its benefits are widely understood and adopted.

Hype Cycle for Artificial Intelligence, 2022



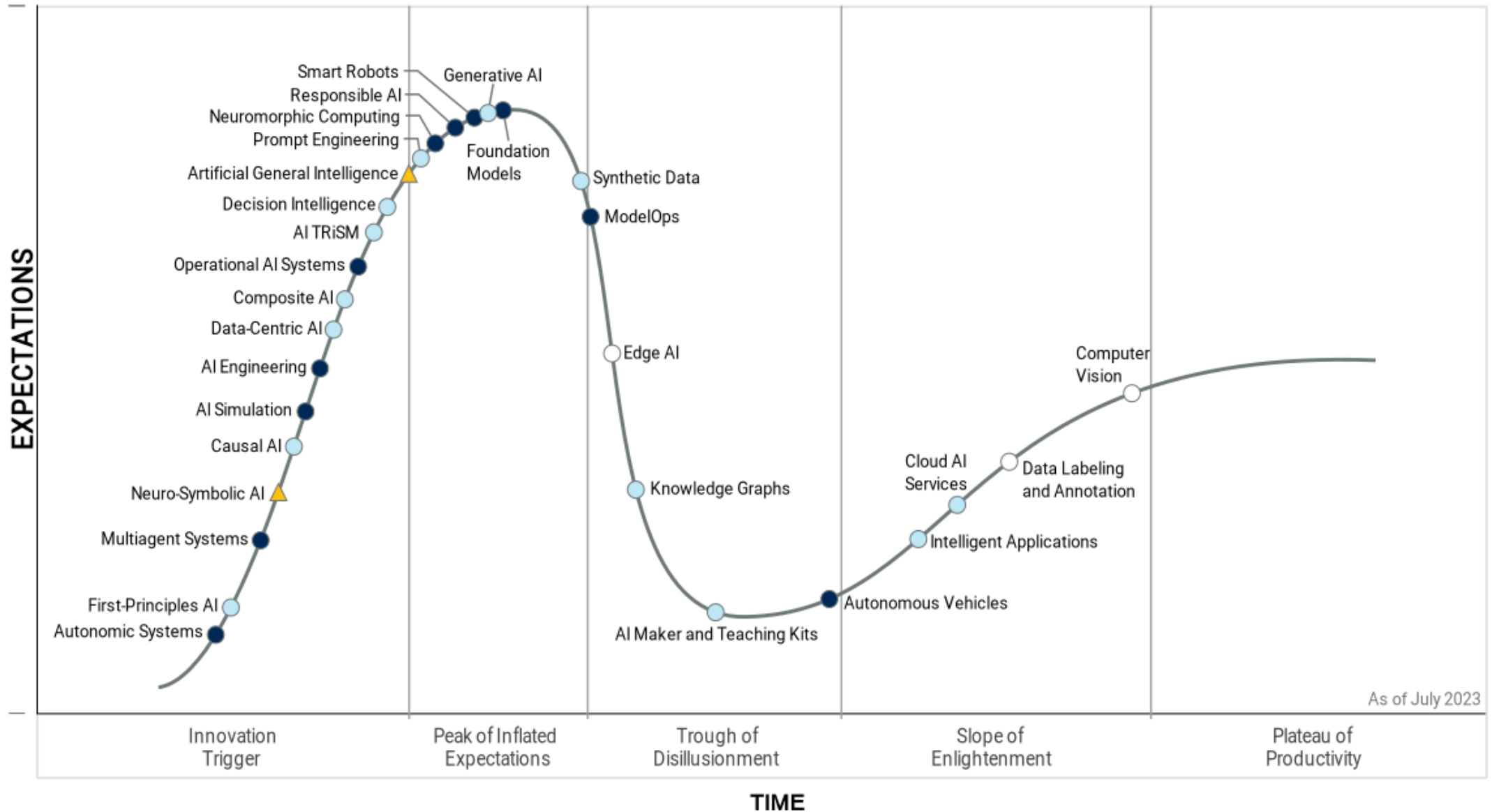
[gartner.com](https://www.gartner.com)

Source: Gartner
© 2022 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner and Hype Cycle are registered trademarks of Gartner, Inc. and its affiliates in the U.S. 1957302

Gartner®

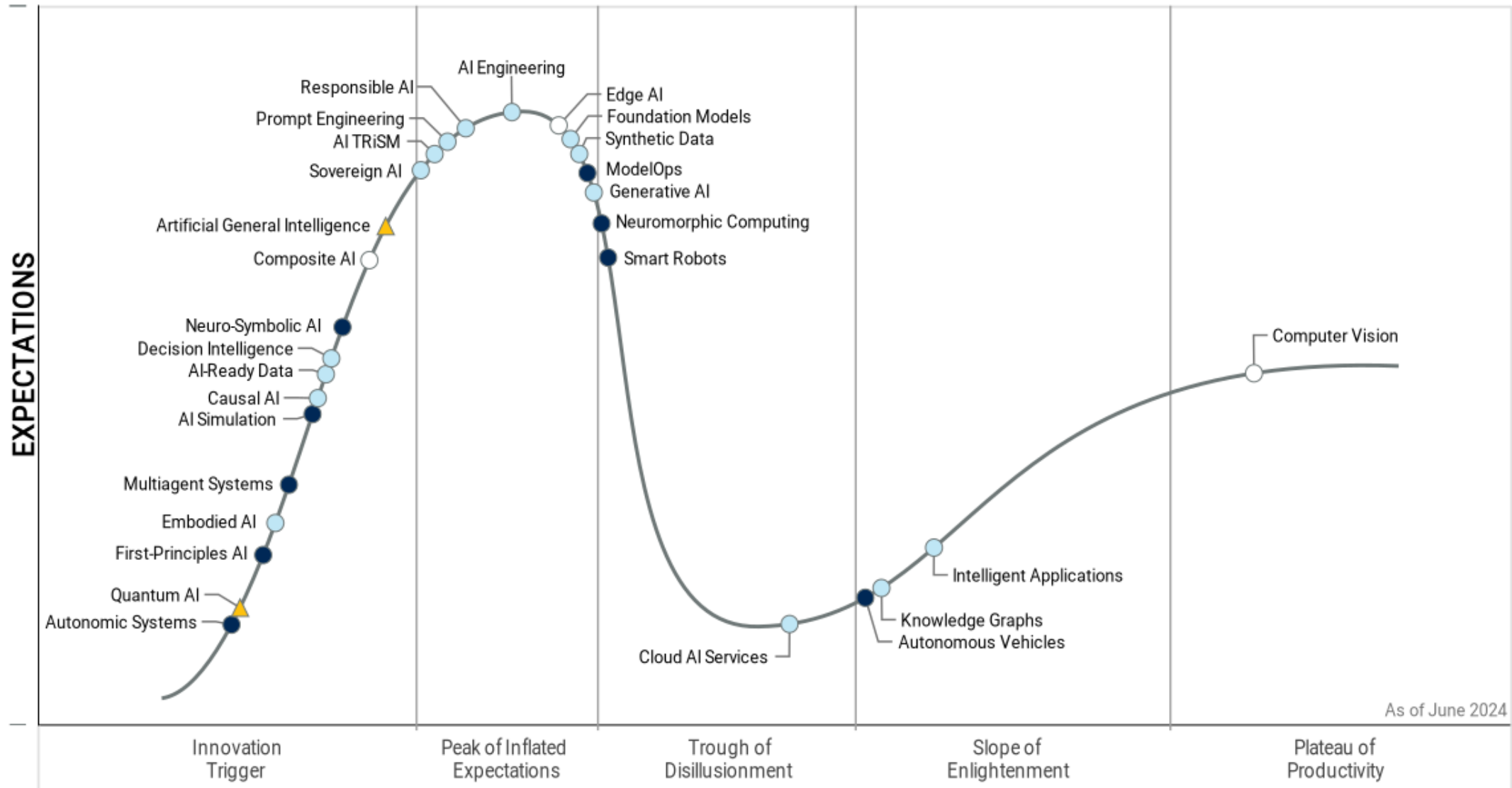
JSU SAN JOSÉ STATE
UNIVERSITY

Hype Cycle for Artificial Intelligence, 2023



Plateau will be reached: ○ <2 yrs. ● 2-5 yrs. ● 5-10 yrs. ▲ >10 yrs. ✗ Obsolete before plateau

Hype Cycle for Artificial Intelligence, 2024



As of June 2024

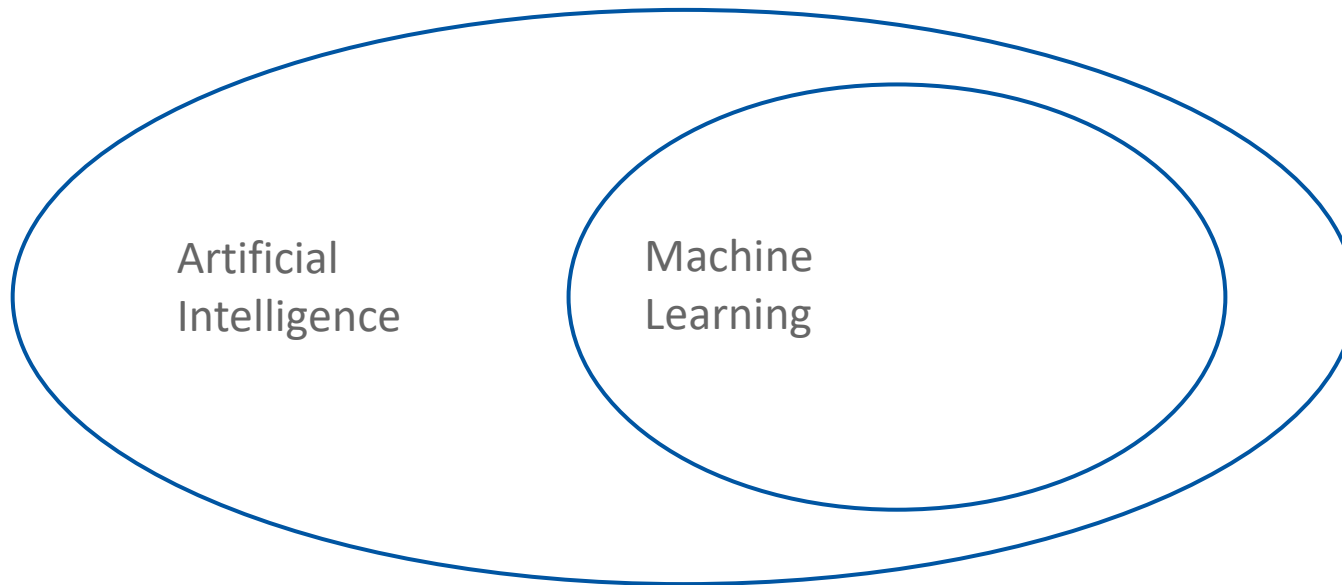
Plateau will be reached: ○ <2 yrs. ● 2-5 yrs. ● 5-10 yrs. ▲ >10 yrs. ⊗ Obsolete before plateau

Front line of LLMs

- Multimodal LLMs (text, image, audio, video)
- text-to-image generation, text-to-video generation
- Fine-Tuning LLMs
- Retrieval Augmented Generation(RAG)
- Multi Agentic LLM
- Edge-AI (deploy LLMs without internet connection)

AI versus ML

“Artificial intelligence (AI)—defined as a system’s ability to correctly interpret external data, to learn from such data, and to use those learnings to achieve specific goals and tasks through flexible adaptation” from Kaplan and Haenlein



AI: Artificial Intelligence

ML: Machine Learning

What Is Machine Learning?

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

—Arthur Samuel, 1959

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

—Tom Mitchell, 1997.

What Is Machine Learning?

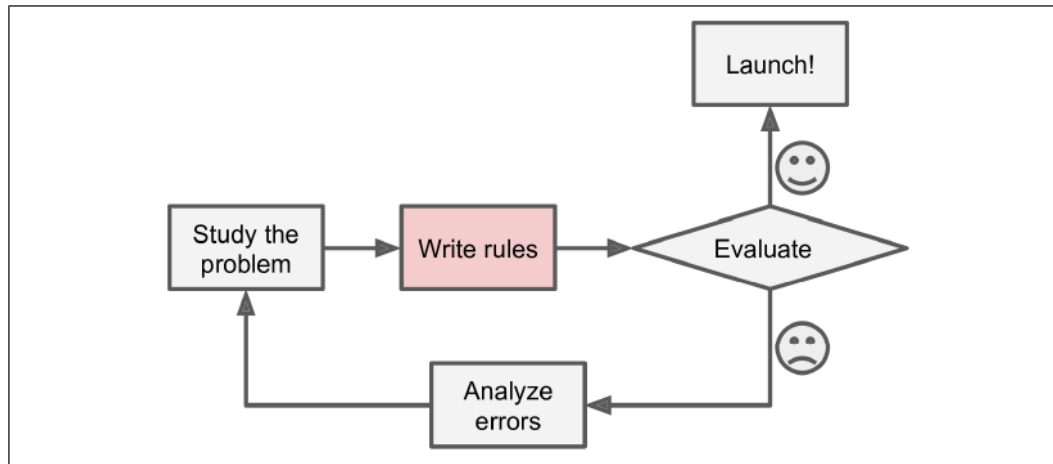


Figure 1-1. The traditional approach

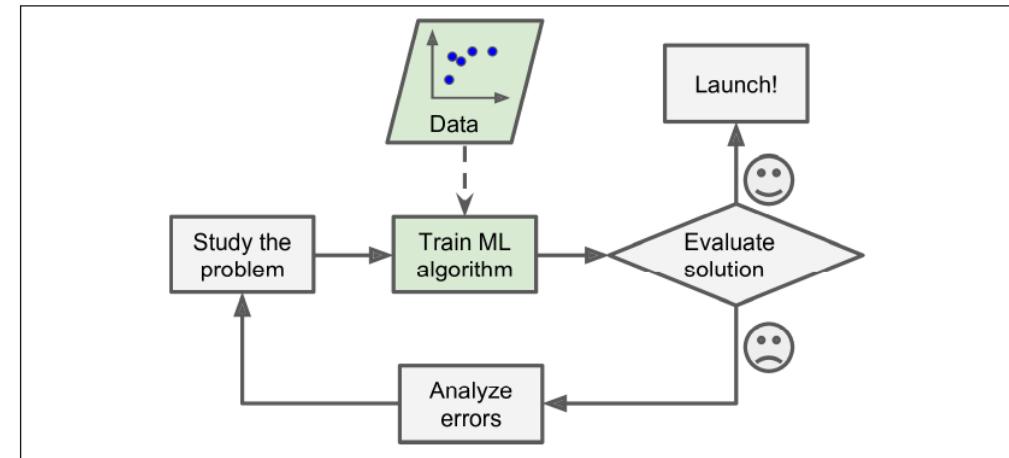
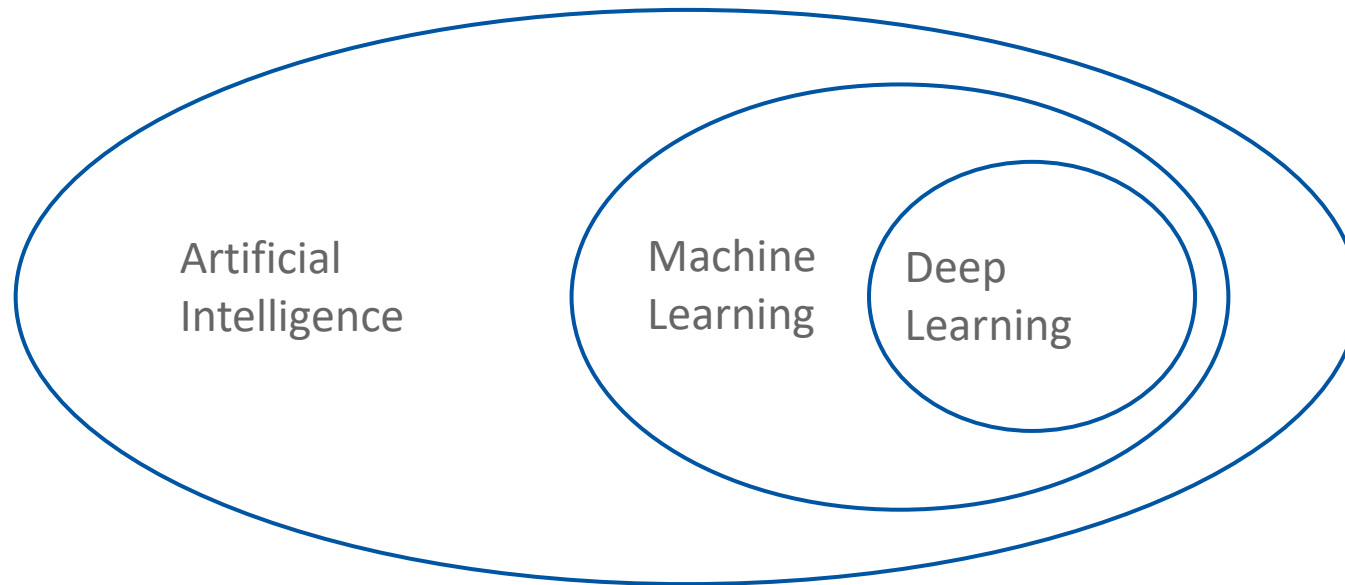


Figure 1-2. Machine Learning approach

Regression vs. Classification in ML

- In a regressor, the model tries to emit a continuous numerical value given a specific input.
- Classifiers have a set of prespecified classes. Given a specific input, they try to emit the class to which the input belongs.

Machine Learning vs. Deep Learning



Biological Neurons

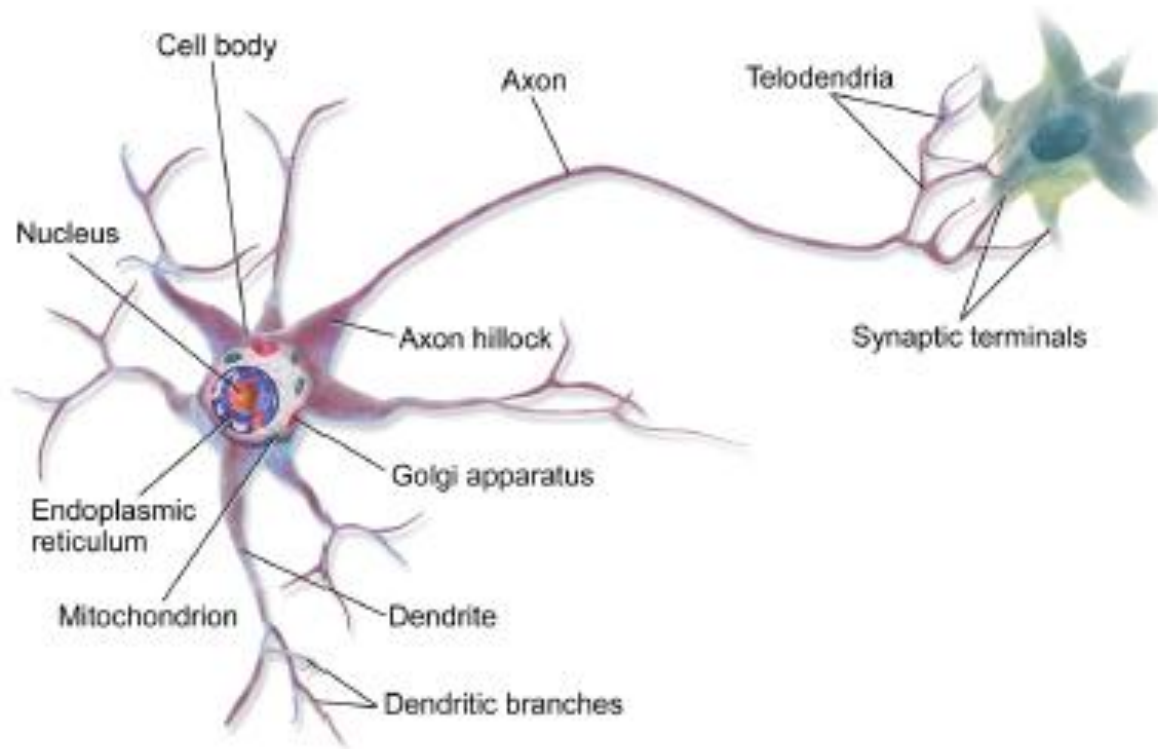
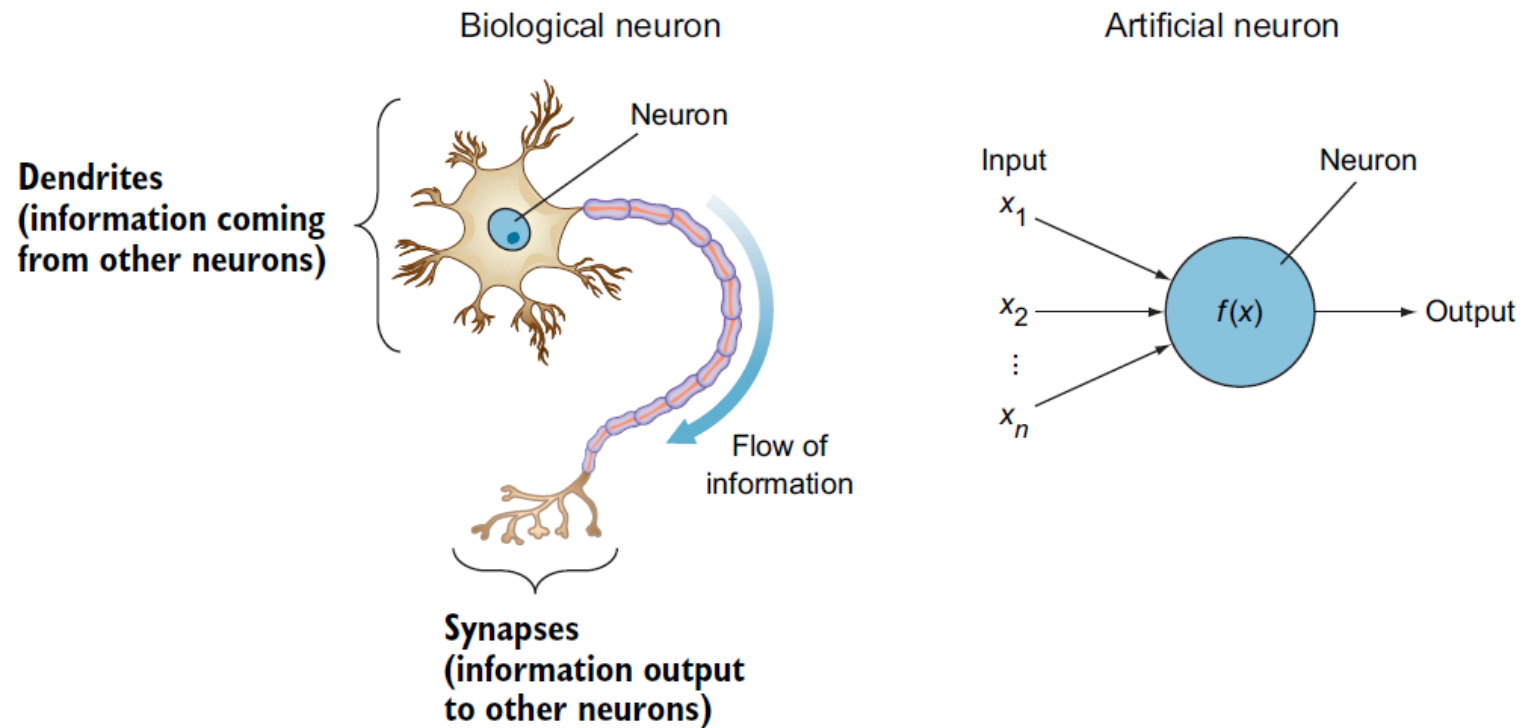
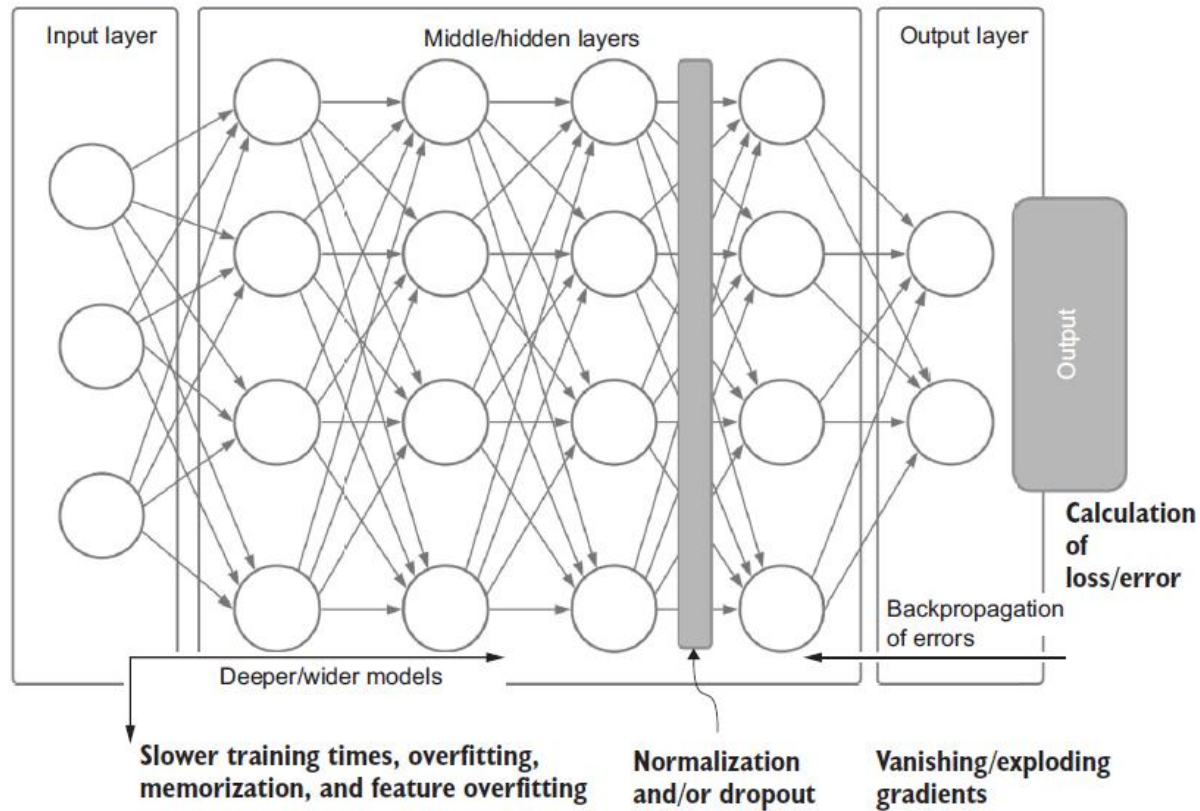


Image by Bruce Blaus ([Creative Commons 3.0](https://en.wikipedia.org/wiki/Neuron)). Reproduced from <https://en.wikipedia.org/wiki/Neuron>.

Similarities between biological neurons and artificial systems



Neural network architecture



Source: Evolutionary Deep Learning, Micheal Lanham

Univariate logistic regression: Two neurons

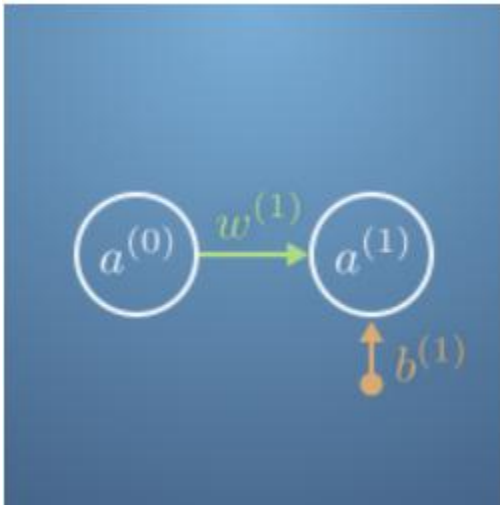


image: Mathematics for Machine Learning

Input layer: x

Output layer:
(first layer) $Z^{[1]} = W^{[1]} \cdot x + b^{[1]}$

$$a^{[1]} = \sigma(Z^{[1]})$$

Activation function: $\sigma(z) = \frac{1}{1 + \exp(-z)}$

Univariate logistic regression: Two neurons

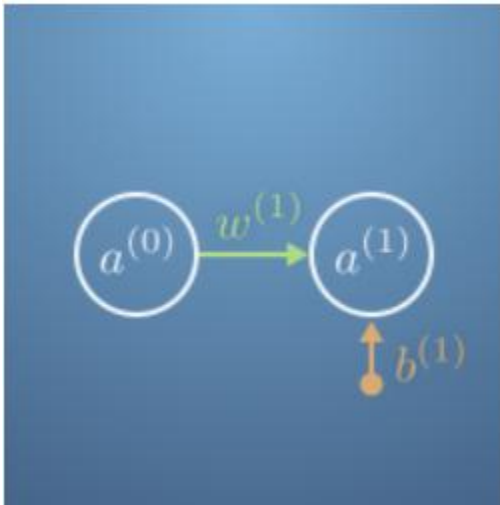


image: Mathematics for Machine Learning

Input layer: $a^{[0]} = x$

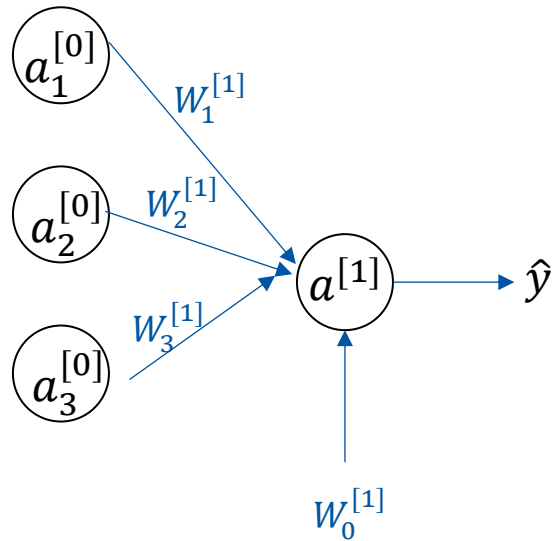
Output layer:
(first layer) $Z^{[1]} = W^{[1]}a^{[0]} + W_0^{[1]}$

$$a^{[1]} = \sigma(Z^{[1]})$$

Activation function: Sigmoid

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Multivariate Logistic Regression

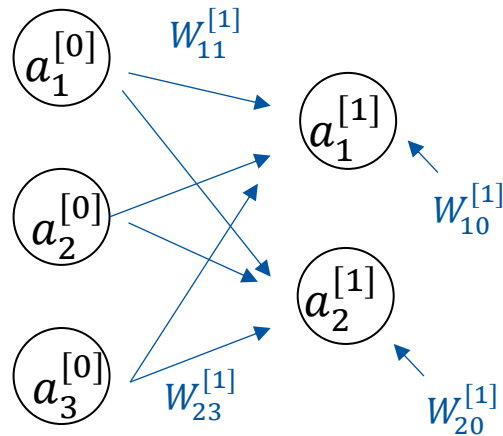


$$Z^{[1]} = W_0^{[1]} \cdot 1 + W_1^{[1]} \cdot a_1^{[0]} + W_2^{[1]} \cdot a_2^{[0]} + W_3^{[1]} \cdot a_3^{[0]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

$$\hat{y} = \begin{cases} 1 & \text{if } a^{[1]} \geq 0.5 \\ 0 & \text{if } a^{[1]} < 0.5 \end{cases}$$

Multiclass classifier



Input layer

Output layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Hidden Layer

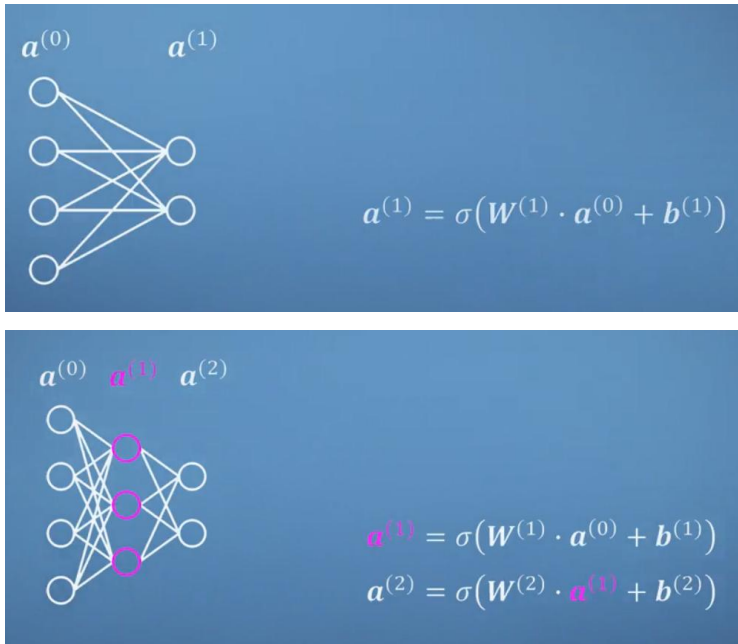


image: Mathematics for Machine Learning

- Another layer (hidden layer) can be inserted between input layer and output layer.
- It works well if there is no direct (or strong) correlation between input layer and output layer.

Neural network

- Machine Learning : No Hidden layer
- (Shallow) Neural Network : One hidden layer
- Deep Neural network: More than two hidden layers

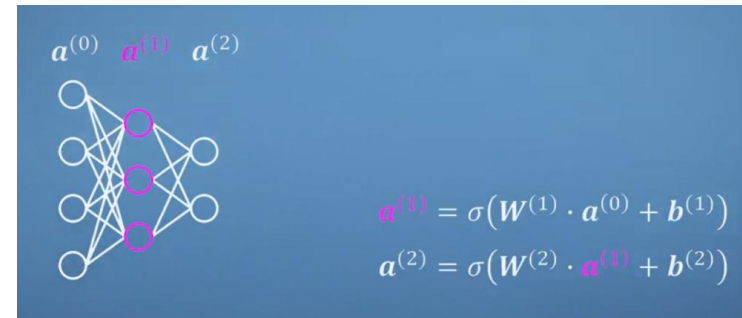
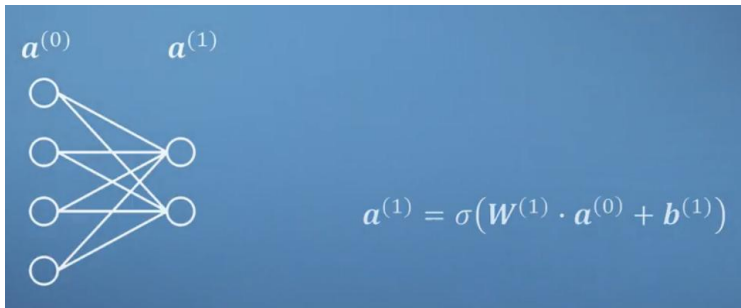
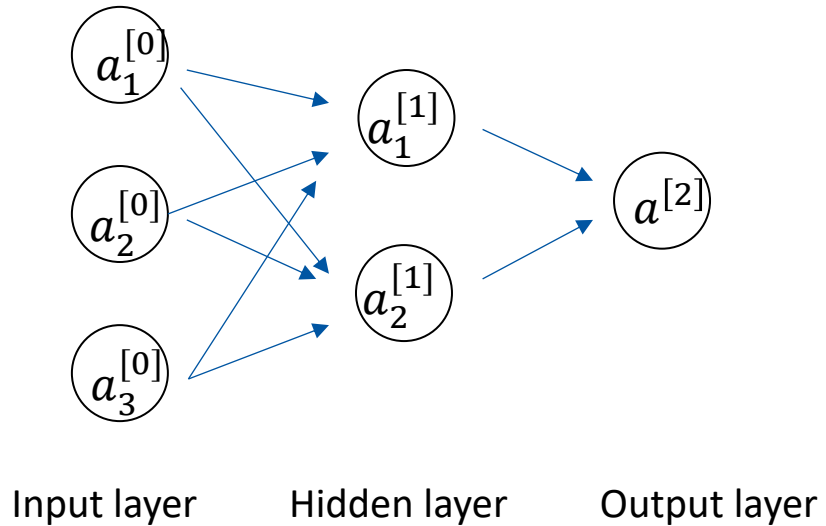


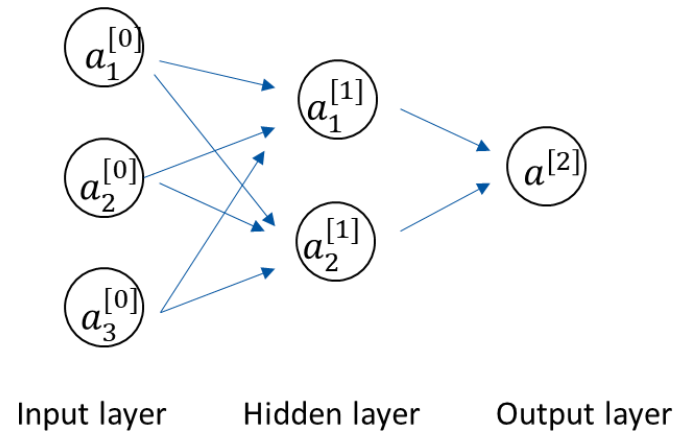
image: Mathematics for Machine Learning

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Shallow Neural Network



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

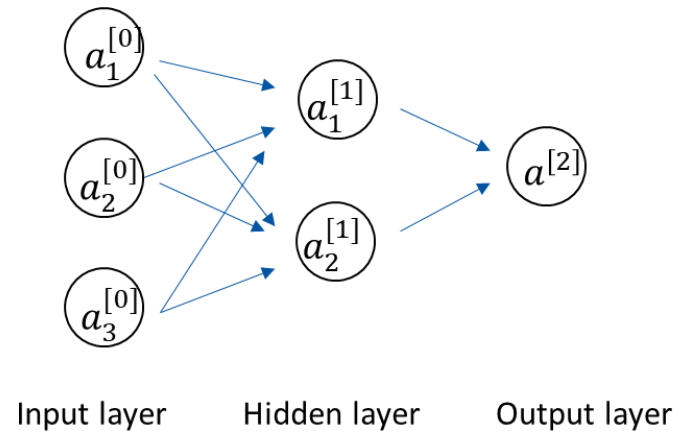
$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

Shallow Neural Network



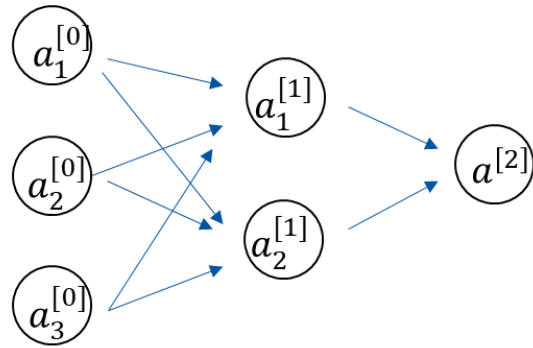
- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

Neural network with matrix form



Input layer Hidden layer Output layer

- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Hidden layer

$$Z_1^{[1]} = W_{10}^{[1]} + W_{11}^{[1]} \cdot a_1^{[0]} + W_{12}^{[1]} \cdot a_2^{[0]} + W_{13}^{[1]} \cdot a_3^{[0]}$$

$$Z_2^{[1]} = W_{20}^{[1]} + W_{21}^{[1]} \cdot a_1^{[0]} + W_{22}^{[1]} \cdot a_2^{[0]} + W_{23}^{[1]} \cdot a_3^{[0]}$$

$$a_1^{[1]} = \sigma(Z_1^{[1]})$$

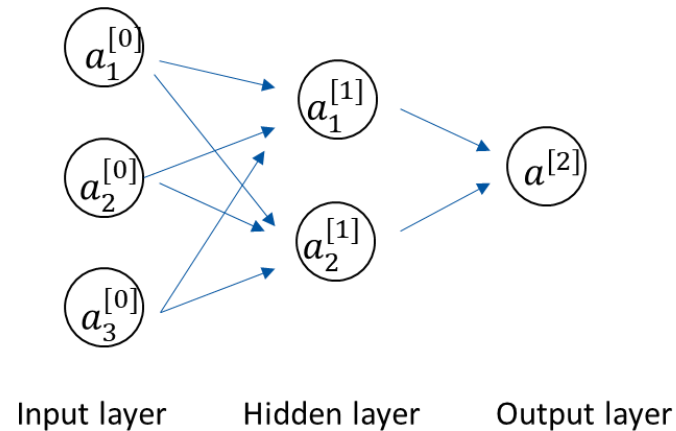
$$a_2^{[1]} = \sigma(Z_2^{[1]})$$

$$W^{[1]} : [2,4]$$

$$W^{[1]} : [n_h, n_x+1]$$

(hidden-node, input-node)

Neural network: matrix dimension



- Input layer : 3 neurons
- Hidden layer : 2 neurons
- output layer : 1 neurons

Output layer

$$Z^{[2]} = W_{10}^{[2]} + W_{11}^{[2]} \cdot a_1^{[1]} + W_{12}^{[2]} \cdot a_2^{[1]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$W^{[2]} : [1,3]$$

$$W^{[2]} : [n_o, n_h+1]$$

(output-node, hidden-node)

Initialize weights

Weights should be initialized randomly rather than to all zeros.

If weights are initialized to be zero, then, each neuron in the first hidden layer will perform the same computation. So even after multiple iterations of gradient descent, each neuron in the layer will be computing the same thing as other neurons.

Weights initialization

Need to break symmetry.

If $W^{[1]}$, $W_0^{[1]}$, $W^{[2]}$, $W_0^{[2]}$ are started with zero,

Then, $Z^{[1]}$, $Z^{[2]}$ become zero.

$a^{[1]}$, $a^{[2]}$ become 0.5.

$dW^{[2]}$ is same

$a^{[1]}$, $a^{[2]}$ will always get same updated!

Small random numbers

$W = 0.01 * \text{np.random.randn}(D, H)$

samples from a zero mean, unit standard deviation Gaussian

Randomness

Randomness shows up in many places: parameter initialization, dropout, data ordering, etc.

For reproducibility, we recommend you always pass in a different random seed for each use of randomness.

Determinism is particularly useful when debugging, so you can hunt down the bug.

```
# set random seed  
seed = 0  
torch.manual_seed(seed)
```